

LLM Agents for Interactive Data Analytics

Project for CS-6604 Advanced Topics

Sandeep Kollipara[†]

Department of Computer Science

Virginia Tech

Alexandria, VA, USA

sandeepk@vt.edu

ABSTRACT

Coding agents today are capable of fully generating projects based on clear user requirements through prompts alone, but the process gets complicated when data analysis is involved. Recent work in LLM Agents has seen some success with Spyder-2v [5] and Data Interpreter [6] having achieved Success Rate (SR) in lower double digits for Data Science and Engineering workflows. Moreover, the performance drops considerably when new data outside its training set is introduced to LLM.

This project aims to develop a black-box analyzer that accepts a file containing tabular data and data description information to set up context for interactive data analysis with the user in loop. In the event of failure, it provides visibility and control to the user fully to revert, modify and continue the task onward. The focus of the project is to automate the initial stages of Data Analytics lifecycle through LLM agents based on analytical results and not just trained memory. To this end, agents are designed with prompt templates and tools to leverage the attention and reasoning capabilities of LLMs effectively. The project code can be found at the following GitHub link:

www.github.com/sandeep-kollipara/cs6604-advanced-topics-in-ai and the video demonstration of the application uploaded on YouTube here:

www.youtube.com/watch?v=5opp5y7zkTM

1 INTRODUCTION

AI Co-pilots and LLMs have demonstrated tremendous potential for automation in software development. By training on codebases for every available framework and programming languages combined with the evolved attention span of the latest generation, they can generate code for complex projects from a simple prompt by the user. The same could not be said when it comes to data analytics processes. Data analytics projects rely heavily on the nature of features and their distribution. Currently, the data must be self-analyzed by the user before requesting explicit actions through prompt. This imposes tedious work for the user in framing prompts with information regarding the data itself. The proposed method alleviates the manual intervention in the preprocessing stage of data analytics using LLM agents.

2 CONTRIBUTIONS

This project will test the effectiveness of Human-In-The-Loop workflow with segmented LLM roles at different stages of the data analytics pipeline. The algorithm involves automation of the most common techniques involved in preprocessing stage. The developed application should enable even a naïve user to perform analysis on any data file on hand with simple text commands.

3 RELATED WORK

Notable recent works are Spyder-2v [5], Data Interpreter [6] and DataMorpher [1]. The former 2 are multimodal LLM agents aimed at tackling Data Science tasks while the latter an LLM for automating data management workflow. For the latter, sample data from DDL files was provided within prompts for the LLM to generate SQL code as output. When it comes to accomplishing tasks, the Success Rate (SR) is still considerably as low as 14% for Spyder-2v [5]. A benchmark paper MAgentBench [2] also points out to a sharp decline in accuracy when the LLMs are faced with new data. The new CodeAct architecture [4] designed by incorporating the code correction algorithm developed by AlphaCodium [3] is likely the next step in development of LLM assisted code agents and possibly data analytics.

3.1 MOTIVATION

With the emergence of agents interfacing with external tools, there is a possibility of solving the complex data science tasks through LLMs with a little assistance from the tools and the user themselves. This project aims to do so by focusing on the initial stages of Exploratory Data Analysis (EDA) and Data Preprocessing while keeping the user in loop. The process follows an interactive path to the goal with checkpoints for the user step in for review and feedback.

Can agentic tools and user feedback guide an LLM to successfully accomplish data analytics tasks?

3.2 PROBLEM SIGNIFICANCE

The reasons for failure by the LLMs to accomplish data science tasks can be attributed to the generalizability of the LLMs. Without context, they return generalized solutions that may not

apply for the current dataset. While the requirements alongside the prompt are indeed considered, there is a context limit of 15 actions per response for a multimodal agent like Spyder-2v [5]. Hence, context is to be managed here.

4 PRELIMINARIES

While there are many depictions of data analytics or machine learning lifecycles, they can be broadly divided into two halves based on the focus of the process: data and model. In general, the first half involves framing the problem or the business understanding through visualization followed by data cleaning and preprocessing steps involving feature selection, extraction and engineering. This is a divergent and decision-intensive task dependent on in-depth statistical and domain knowledge whose outcomes directly affect the next steps involving the model, namely model planning, building and deployment.

Returning to the original premise on coding agents, their contribution to the data science tasks is mostly in the second half with the user having already completed the data analysis and model planning. While this approach reduces the burden of additional work on the LLM, it requires the user to compensate by providing sufficient context for it to carry out the next tasks. This is essential for the LLM to tailor the code generation for the data of interest. This further extends and complicates the user prompt in comparison to a software development task. Another significant limitation lies in conveying the nature of higher dimensional data such as data in multimedia formats like images, audio, video, etc. through prompts. itself.

5 ALGORITHM

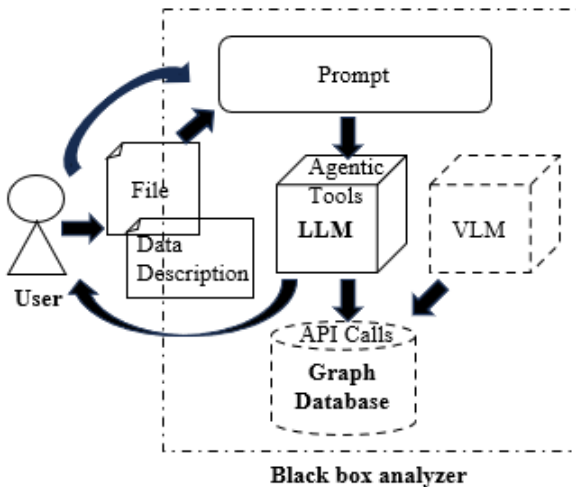


Figure 1: An illustration representing flow of the interactions between the user and components of the black-box analyzer

This section explains the development of the application encasing the LLM agent and associated components necessary for end-to-

end interactive data analytics, named black-box analyzer, which is the proposed idea to the research problem. The process flow diagram in Fig. 1 depicts the primary subjects: LLM, User and the Graph database exchanging data in the form of files, prompts, agentic tools and API calls. One cycle of data exchange between the components is briefly depicted below:

1. User provides the file alongside data description information to the application which generates prompt using a template to set the context. From the second cycle onwards, the user must only confirm options or query back in text.
2. The LLM always generates a response in the form of text which is directed back at the user in conjunction with an API call to the graph database and/or access its agentic tools for some action. The collated results from the action combined with the text response are sent to the user for review and feedback alongside options for the next steps.

There are only two steps overall, one each from the User and the LLM (driver of the black-box analyzer). We define their roles and actions more clearly in the next sections while also expanding on the supporting components later. The main function of the supporting components is:

1. Design interactive prompts based on data preprocessing steps before model implementation.
2. Use agentic tools that can peruse through data description information and design a pipeline to apply customized data standardization.
3. Store the queries, actions and responses in a graph database.

5.1 Role of LLM

The LLM is the central component of the black-box analyzer. With the generalized nature of application, it can be leveraged for tasks requiring specific domain knowledge. Its attention span allows the scope to fit the problem while its reasoning gives it the decision-making ability. Its implementation in the black box analyzer is manifold, as mentioned below:

1. **Code Generation:** The primary function of the LLM still remains as a coding agent. The prompt template will provide the requirements including the sample data and feature information and elicit generation of data-compliant code. The environment provided via agentic tools will provide handlers to run the generated code and return the results, textual or graphical.
2. **Variable Detection:** The categorical variables usually belong to one of the two types: ordinal or nominal. While the former can be encoded directly with numeric data, the latter cannot be encoded so and must alternatively be one-hot encoded which increases the dimensions by the number of categories (minus one). This detection in case of unknown input data can improve model building by reducing the curse of dimensionality for ordinal types.
3. **Domain-specific Feature Glossary:** The descriptions of the features can be generated for unknown data based on the analysis of the contents. This can be compounded with any

pretext provided by the user regarding the domain of the input data to more accurately guess the feature. This is especially meaningful when eliminating features within the same ranges, the model can choose the explainable feature over the other.

4. **Filter method Thresholds (*):** Many of the feature elimination methods involve enforcing a filter on a metric such as variance or correlation amongst the features. These filters can be applied flexibly depending on the model as well as the features in question. Important features can be highlighted to the user for feedback with suggestions to alter the threshold or make an exception.
5. **Decision between close outcomes:** Many important features could be eliminated in the intermediate steps of model building for a slight increase in accuracy at the cost of interpretability, due to nature of model algorithms. These refinements could be rechecked with shuffled samples or highlighted to the user for reconsideration. Most of these cases will be handled in reference to a metric from the p-tests of the sample's distribution.
6. **Summarization (*):** The changes to the data can be summarized for context management. This is a countermeasure in response to eventual loss of context after set number of actions/cycles. Summarizing the history and storing in a graph data structure enables tracking and reversal of steps, in case of failure or if asked by the user.
7. **Graphical Analysis (*):** Vision Language Models (VLM) can provide image-to-text description of graph and other visual outputs to user or take actions based on the result. Standard large language models cannot do this effectively. As such, this option is considered out of scope for this project at present.

The features marked by (*) were not implemented in the final version due to restriction of scope. Most of the above actions will be conveyed through carefully engineered prompt templates and agentic tool access, handled by a backend application utilizing an object-relational mapping approach in conjunction with graph database.

5.2 Role of User and HITL Workflow

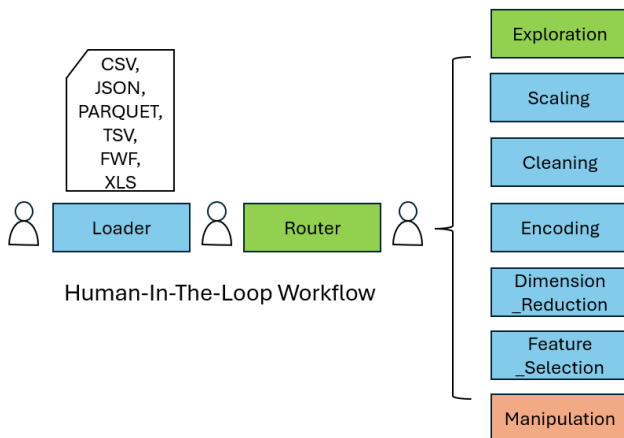


Figure 2: HITL Process with different LLM roles

The user is the judge with the ability to override, redo, revert or modify the LLM's suggestions. To decide, the user needs a static reference (control) to compare the LLM's suggestion (experiment). A full-fledged rule-based algorithm will be generated for a dataset at the beginning of the session, and subsequently for every action that LLM takes. It will encompass the generally available. The agents utilized in this project are simple and non-collaborative with their function dedicated to proper execution of tools based on user input. This is due in part to the Human-In-The-Loop workflow described below.

Each of the roles of the LLM indicated in Figure 2 describes a section of the analytics pipeline with its own toolset. They are inherited from the agent class with their own properties and methods defined, with the sole exception being the 'Router' whose purpose is to redirect the user query to the corresponding agent suitable to the task. Also, the 'Manipulation' process is described but not implemented as part of this project. So the graph database for the persistent storage of transition states within the program. However, the interactive 'rewind'/'fast-forward' functionality replaces this for fully controlled huma-in-the-loop experience.

5.3 Rule-based algorithm

The rule-based algorithm is the conventional format of data analysis up until the model building phase, with every aspect of data interaction framed as an action step. The inputs of the process are the individual features of the dataset defined from the flat-file and data description information (conventionally the raw CSV or delimiter-separated file format with a data definition language file respectively).

The first discovery phase encompassing Exploratory data analysis (EDA) and Visualization select the current dataset and only convey graphical and text-based results to the user. The user can access these steps at any point with no lasting changes to the data itself.

The second preprocessing phase involves several operations applied conditionally based on the type of feature. The data cleaning is conducted based and null values filled based on mode for categorical and mean for numerical by the sample. The data encoding is done by the subtypes of the above as follows in reference to Figure 3:

1. **Numerical:** This type is further subdivided into 2: a) Discrete and b) Continuous with the latter having further 2 subtypes: i) Interval and ii) Ratio.
2. **Categorical:** This type is classifiable into 2 types: a) Ordinal and b) Nominal.

This is followed by implementing dimensionality reduction through Principal Components Analysis (PCA) or Linear Discriminant Analysis (LDA) based on whether the data has a target that is not labelled or labelled respectively. Finally, the best features are selected based on the model to be built. This includes χ^2 -test for classification and t-test for regression. The Feature

Extraction is left for user definition due to the sheer variety of options available for testing and hence, excluded from the rules.

The expectation for the LLM agent is to assist the user with preprocessing from a raw dataset to a model-ready latent space through interactive user guidance. The tools should conveniently pass the data to the environment and effectively act as a black box with only the prompt being available to the user.

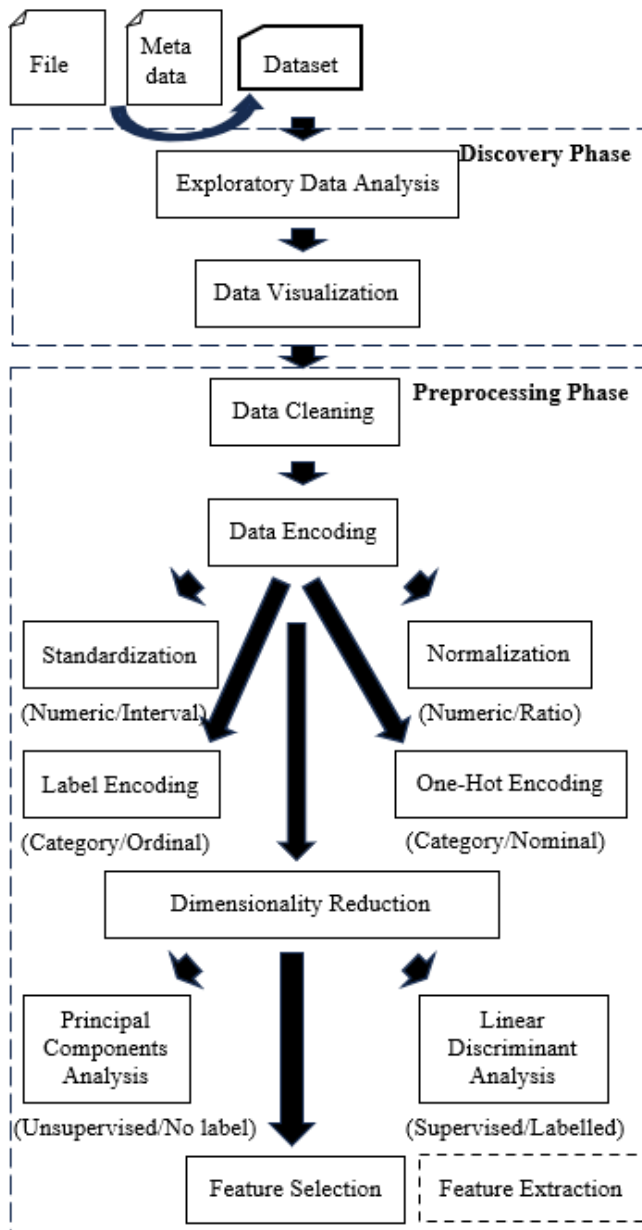


Figure 3: A process-flow diagram of the rule-based algorithm coded as an alternative option for the user.

5.4 Software Design and Implementation

The Object Model Design was adopted in anticipation of parallel development of multiple agents designated with a role and/or given access to an assortment of tools. There are multiple features across agents that need to be considered whilst designing the common backend of the ReAct agent functionality. Some of these features include the user chat function, file I/O, context management, function/tool access etc. By using the object model design, all these are modularized into well-defined classes with customized tools available to agents.

The first class that was designed acts as the foundation for building all the others: it encompasses a simple **ReAct agent**. By default, it has no functions ‘bound’ to it but can receive new tools during initialization as an argument. It only serves to provide a connection to LLM. It exposes only a public call method that receives the user input and responds back in text with or without a function call.

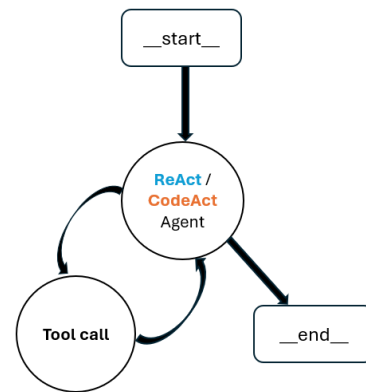


Figure 4: The workflow of a ReAct (Reasoning + Action) agent used in the deployment of various roles of the LLM. An alternative called CodeAct was studied but not implemented.

With developments in the AI field occurring at a lightning pace, latest models and with new specifications replacing old ones, conforming to a specific LLM integration framework is unwise as their API are being deprecated rapidly. It is prudent to define a class encapsulating the framework that can be updated or replaced as necessary in the long-term. By decoupling the LLM frameworks, the features and agents can be developed independently without relying on any specific aspect of LLM frameworks. This also allows us to use multiple frameworks in conjunction if they share the defined methods of the classes. The following agents were developed with inheritance (subclass → superclass):

1. **Base Agent (→ ReAct Agent):** This agent holds the data structures common across all the agents. This includes the data itself, chat history and data description summary. It automatically inserts data description at the end of every user query providing context while enhancing response accuracy. At present, only one file per instance is supported and loading another file overwrites the current one.

2. **Router Agent (→ Base Agent):** This agent is the central piece of the framework. Its role is to receive the user query that either commands the state transition in the program or calls the other agents to handle a specific task. It discerns the intent of the user through tagging or classifying the query under one of the labels below which is then programmatically detected and executed:
 - 'end': End the program
 - 'fast_forward': Go to the next state
 - 'rewind': Return to previous state
 - 'stall': Stay at the same state
 - 'load_save': Loading/Saving task
 - 'exploration': Data Exploration task
 - 'cleaning': Data Cleaning task
 - 'scaling': Data Scaling task
 - 'encoding': Data Encoding task
 - 'dimension_reduction': Dimensionality Reduction
 - 'feature_selection': Feature Selection
 - 'manipulation' (*): Data Manipulation
3. **Loader Agent (→ Base Agent):** This agent is the first to interact with the user before relaying that role to the Router Agent. It is responsible for loading and saving files from the designated directory. It incorporates Vector Similarity Search in finding the filenames in the directory and returns the file with the closest match, thereby giving leniency to the user in providing only an approximate filename. In addition, it takes an additional text input, usually the data or attribute description of the file, which is then appended to the context of every user query henceforth. If the user wishes to save the current preprocessed data or load another file (overwriting the current), they may request the Router Agent to call this agent anytime in future.
4. **Cleaner Agent (→ Base Agent):** This agent is called upon to perform data cleaning tasks. There are multiple data cleaning functions related to null value treatment, outlier treatment, redundant feature elimination available that can be called with a threshold or filter argument. All tools will be executed with default parameters unless specifically mentioned. Due to binding a higher number of tools than the others, this agent may not invoke the tool on the first call and needs some reassertion.
5. **Scaler Agent (→ Base Agent):** This agent filters the data for numerical features and then applies data scaling techniques which are standardization or normalization depending on the distribution of the data. This is further explained in greater detail in the Reasoning Scenarios section.
6. **Encoder Agent (→ Base Agent):** This agent filters the data for categorical features and then applies data encoding techniques depending on the feature being an ordinal or nominal type. If it is ordinal, it applies label encoding to the feature based on the ranking or hierarchy in the classes. It applies One-hot encoding in the other scenario. The tool design for this agent is covered in detail in the Reasoning Scenario section
7. **Dimension Reduction Agent (→ Base Agent):** This agent conducts dimension reduction techniques such as Principal Component Analysis (PCA) for unlabeled data and Linear Discriminant Analysis (LDA) for labelled data. For PCA, the tool design also considers the feature reduction logic which is explained in the Reasoning Scenarios section.

8. **Feature Selection Agent (→ Base Agent):** This agent performs feature selection either through manual selection or via techniques such as Random Forest Analysis and Backward Stepwise Linear Regression. The manual selection is applied either by selecting or dropping the feature names provided in the user query. To assist with the chat, the user can approximate the feature names which are then referred to the columns present in the current data and mapped automatically. This is carried by providing the LLM agent with the column details of the current data by appending them to the user query. The Random Forest Analysis tool
9. **Explorer Agent (→ Base Agent):** This is Pandas DataFrame Agent, a framework agent from Langchain library that can operate on the provided data using the Pandas library functions. It can generate code based on the user query and running it on the provided data and can do most data related tasks with the downside of not returning the data back after said user query. Hence it is not usable for tasks involving data transformation or manipulation as it can only return the analysis and plots. This makes it perfect for Exploratory Data Analysis and Visualization which does not modify the data in any manner. Also, this is currently the only agent capable of code generation with the exception below.
10. **Manipulation Agent (*) (→ CodeAct Agent):** This agent is not implemented in the current version. It was based of CodeAct agent which can generate and run function code in a virtual environment and return objects back from the sandbox thereby making it the most versatile option available. However, it is still not available with support for complex data structures for tabular data at the time of this report.

The UML diagrams for the tool/function bindings with each of the above agents are depicted in Appendix A. Another benefit of the object model design is the advantage of independent testing of agents except for Loader Agent and Router Agent which are basic dependencies for data availability and user query routing. The order mentioned above follows the development process with the earlier agent usually required for the upstream preprocessing task, for example, feature selection or dimensionality reduction requires the data to be encoded to a numerical format. This requires that data be passed through the Scaler Agent and Encoding Agent beforehand. Exception handling was also carried out at the class level as well as function level where any error encountered would not result in the program malfunctioning, but rather the agent receives this and relays this to the user as part of the chat conversation. This design pattern has allowed the development process to parallelly focus on the tools and templates as well.

5.5 Template Design

The prompt templates are the instructions provided at the initialization of the LLM agent before the start of conversation. Sometimes termed as system prompts, they instill a premise where the LLM adopts either a role or learns the rules for executing the query. There are two types of template patterns which are used for: 1) Tagging and 2) Task Completion.

The former is basically used for classification of the user query to one or more of the tags, useful for ascertaining the nature of task or action described by the user query. The latter is for execution of tool calls with added instructions based on the output from the tools. This is where we implement the reasoning scenarios for the tasks. In addition to running the tools, the agents can analyze and reason through the results of function calls to take subsequent actions or make assertions back to the user.

The prompts were designed for every agent except for the ReAct Agent, Base Agent and Manipulation Agent. Most fall under Zero-shot Prompting with specific reasoning scenarios using One-shot. The experimentation with prompt templates remains an unexplored avenue as much of the time for testing went into toolset/function design.

5.6 Toolset/Function Design

The tools/functions form the most comprehensively explored and tested portion of the project with primary focus on their evaluation for reasoning scenarios. Yet, there is a vast scope of improvement in their design. Tools are not just functions that execute the code for the task, they should also communicate their progress clearly to the overseeing LLM Agent. Proper design of tools can allow LLM agents to work with feedback and retry until the goal is reached. This requires that the tools be annotated clearly with descriptions for their function, arguments and outputs.

The functions used for data handling and preprocessing were defined separately with a focus on accurately performing the data related task. They were imported and wrapped within class methods to redefine them with arguments suitable for the LLM to call. This also allows the functions to implicitly access class variables including the data held by the agent without having to pass them as explicit arguments. With this design, the data-oriented functions can be redefined as LLM-oriented tools.

However, the tools cannot manage complex row-based operations on data without accessing it directly, like the Pandas DataFrame Agent (Explorer Agent) or CodeAct agent utilizing a sandbox environment. This is a drawback that cannot be addressed with the current implementation using a ReAct Agent. Also, tool design varies heavily across implementations and underlying LLM models, with the specifications held privately due to patents or trade secrets especially by the best performers in commercial space.

5.7 Reasoning Scenarios

To fully leverage the power of LLM through reasoning and generalizability in data analytics workflow, four scenarios were crafted to involve the agent in analyzing the results of a data related task and acting optimally based on the results. This is heavily dependent on the tools of the agent using which it can examine the current data (functions/tools are limiting the success rate as seen in ‘Results’ section) The exact implementation of the reasoning scenarios is illustrated in Appendix C.

5.7.1 Scaling features based on normality test

The first step in preprocessing the data involves scaling the numerical features to be more consistent and interoperable with respect to machine learning models. The common method is to center the data around its mean and scale it with a standard deviation of 1, this process is standardization. It is best applied to features possessing a normal/gaussian distribution. Another approach is to scale by limiting the maximum and minimum of the data between 0 and 1 respectively, this process is normalization or min-max scaling. It is best applied to non-gaussian distributions and specific deep learning models. It assists with faster convergence and mitigates the problem of vanishing gradients. There are other approaches such as robust scaling that addresses outliers in data, log normalization for skewed features.

For the scope of our application, the options are limited to either standardization of normalization based on the underlying distribution of the data which is ascertained through the Normality test. It tests the null hypothesis that the sample comes from a normal distribution and based on the p-value. The distribution of the feature is determined to be gaussian/normal if p-value is lower than 0.05 thereby rejecting the null hypothesis and vice versa.

The Scaler Agent receives a request for data scaling from the user query (via Router Agent) and unless specified with the exact method of scaling and target features, it applies the normality test to all numerical features in the current data. It receives a dictionary mapping the features to their p-values used for splitting the features based on their nature of distribution. The group with gaussian distribution is then scaled using standardization and the other using normalization. This is also conveyed back to the user in a verbose manner in case any manual changes are to be made or reverted later.

5.7.2 Encoding features based on ordinality

While scaling processes the numerical features to be model-ready, the categorical features undergo encoding to be interpretable by most of the machine learning algorithms which do not support non-numeric datatypes as inputs. There are two methods used for encoding categorical features: 1) label encoding, and 2) one-hot encoding. The former is used for categorical features of ordinal type with an inherent rank or hierarchy in their classes/labels while the latter for nominal types. This was traditionally only ascertainable by a human familiar with the data description. But with LLMs and their generalized knowledge base, this task can be handled by the LLM albeit with a little support in setting up the context by the user. The context here is crucial as tested and proven in the ‘Evaluation Plan’ section.

The Encoder Agent receives context in 2 inputs, first via the ‘memo’ (abbrev. for memorandum) parameter provided during data loading by the user detailing the data description of the file and any associated context with it. The second and more direct approach is through a dictionary mapping the features to all the unique labels (limited to max of 15). While the first input sets the context about the dataset and its domain and nature, the second

gives access to the contents of the feature to the LLM which is then used for determining the ordinality of the feature. The prompt template also instructs the LLM to explain to user the detected order/rank if the feature were to be considered an ordinal type.

5.7.3 Feature selection with Random Forest

The Random Forest Analysis is an entropy-based feature selection technique which can be used for determining the most useful features for predicting the target. The Gini Importance metric, sometime referred to as Variable Importance or Mean Decrease Impurity (MDI), is calculated for each feature with the higher value for a more contributing feature. However, this metric's accuracy assumes that none of the features are correlated. In practice, the analyst working with data can choose any arbitrary number of features with the highest variable importance in relation to insights from data description or other analytical results.

The Feature Selection Agent receives a request for feature selection using random forest from the user (via Router Agent) and only applies the selection if explicitly asked, otherwise it only returns the results of random forest analysis alone. It receives a dictionary mapping all the features to their variable importance metric in descending order. Based on its initialization prompt, it applies first-order differencing on the array of floating-point numbers representing the variable importance metric. It is instructed to segregate the features from the top all the way to the point of the highest decrease in the metric (represented as the maximum difference between adjacent features) and return the selected features as the suggestion. This is a significantly complex reasoning task for the LLM as it often conflicts with its generalized memory demonstrated by the results as seen in the 'Evaluation plan' section.

5.7.4 Dimensionality Reduction with PCA

Principal Components Analysis is a feature extraction technique involving dimensionality reduction. It transforms original features into new uncorrelated components that capture the most variance in the data. Since it is essentially a feature transformation, it erases the original descriptions of the features but allows us a glimpse into the most useful features in the data by evaluating the proportion of Explained Variance metric. This is useful for selecting features that contribute the most to the variance. While the applications are 2-fold as a feature extraction and selection technique, the latter is chosen as the scenario for reasoning.

The Dimension Reduction Agent receives a request for a dimension reduction in data using PCA. If not asked specifically for the number of components, it applies the maximum possible, which is the total number of features minus one. This is essential for our scenario of feature selection. The agent receives a dictionary mapping of features to their cumulative explained variance calculated after sorting in descending order of individual explained variance. The top features are selected by applying a cutoff with the cumulative variance below it.

5.7.5 Miscellaneous untested scenarios

For feature selection, backward stepwise linear regression was implemented with a reasoning scenario analogous to that of Random Forest Analysis with the metric for reasoning being the Adjusted R-squared for a set of features (not just one feature) fitted to the model. Since features are dropped iteratively in backward stepwise linear regression, the difference in Adjusted R-squared is calculated across adjacent iterations. A dictionary mapping the dropped feature to the Adjusted R-squared in the same order of iterations is received by the Feature Selection Agent. It then proceeds to drop all the features before the iteration with the highest difference in Adjusted R-squared. Despite implementation, the results were inconsistent and inaccurate. The root cause analysis suggested that the agent could not execute instructions when paired with another complex reasoning scenario such as Random Forest Analysis.

Linear Discriminant Analysis (LDA) was also fully implemented within the Dimension Reduction Agent but remained untested due to the necessity of a categorical target variable. All the other scenarios required no target variable or a continuous one (numeric). Due to LDA being applied late in the process, it required significant preprocessing in addition to acquiring datasets with a compatible target variable.

6 DATA DESCRIPTION

About 30 datasets were selected from Kaggle's public dataset domain conditional on being suitable for regression analysis, i.e., the target label must be continuous. They were picked randomly across different domains ranging from academic research, commercial sales, real estate sales, geospatial data, biological indicators, electronics testing data and healthcare.

Two variants of the data were created: 1) original/known dataset and 2) hidden dataset. The original as the name suggests has the same column names and data content. The hidden dataset has all features relabeled to abstract name of 'feature<number>' where the number was cycled across all the features except for the target column which was labelled 'target'.

Due to limitations arising from the functions/tools, some of the datasets could not be scaled or encoded due to issues with variable detection. Some of them required manual intervention with respect to removal of ID columns and high non-numeric features before using the evaluation path (mentioned in next section).

7 EVALUATION PLAN

Initially, data from DataSciBench [7] was considered for benchmark testing the agents, but its tasks were too generalized to reliably involve and test the agents and their specific tools designed for them.

Using a set of known and hidden versions (without original feature labels) of the same datasets for context control, we

evaluate the effectiveness of the agents for the reasoning scenarios. While there is no control group in the experiment, we evaluate the performance of both the instances with original and hidden versions of data to test the basis of reasoning on which the decisions are made. Below are the steps taken with each dataset:

1. Load the <file name> with data description (only <filename> in case of hidden dataset)
2. Apply only Missing Value Treatment to the data (not any of the other data cleaning functions)
3. Scale the data
4. Encode the data
5. Conduct Random Forest Analysis with target as <target>
6. Conduct Backward Stepwise Linear Regression with target as <target>
7. Drop the <target> column
8. Conduct PCA on the data

The synthetic datasets validate the LLM-suggested outcomes based on the analytical results alone. If the models are up to par with the ones trained using the known datasets, it confirms the contribution from the prompt-based LLM suggestions. On the downside, a stark difference in the performance between the two indicates that the LLM's suggestions were derived from the memorization from trained data and not based on the analytical results from the context

7.1 Results

Below are the tabulated results in Table 1 for the four reasoning scenarios across known and hidden versions of the 30 datasets with OpenAI's GPT-4o as the API-based model:

Table 1. The success rates for the four reasoning scenarios.

Reasoning Scenario	Original dataset with 'memo'	Hidden dataset without 'memo'
Manual Interventions	17/28 (60.7%)	18/28 (64.3%)
Data Scaling	22/28 (78.5%)	21/28 (75.0%)
Data Encoding	20/25 (80.0%)	13/25 (52.0%)
Feature Selection	21/23 (91.3%)	20/23 (87.0%)
Dim. Reduction	22/23 (95.6%)	22/23 (95.6%)

The manual interventions are actions taken within the program by the user to mitigate the inconsistencies and issues caused by the LLM-assisted actions. These are not because of the data but rather attributed to design flaws of the tools and reasoning of the LLMs. From the results, they are not contrasted across the known and the hidden datasets by much since the tools are incognizant of the data description and work similarly in both cases for the same inputs.

7.1.1 Data Scaling

Data was scaled in 28 instances where numerical features were available within the dataset. The Normality test was executed well for all the cases and the failures were attributed to the function

identifying the numerical features. In all the instances of failure, some of the numerical features were not detected properly due to the flaw in the algorithm. The reasoning scenario was unaffected by the lack of context in this scenario as the decision to scale was not affected by the memorization for any dataset. In fact, the Normality test reinforces the real-world application for the conducted experiment.

7.1.2 Data Encoding

Data encoding results were most contrasting between the known and the hidden datasets as the data description information is most significant in determining the ordinality of a categorical feature. Of the 25 datasets possessing categorical features, 80% of them were guessed correctly when the data description and column names were provided to LLM. In the case of the hidden dataset, only 52% were correctly labelled with the difference attributed to incorrect assumption about the underlying data.

7.1.3 Feature Selection

Random Forest Analysis was bifurcating the feature space quite accurately for most of the cases in both known and hidden datasets. However, the evaluation was lax in not considering the cases where the drop in Feature Importance metric occurs in the very first feature. The LLM seems to ignore the difference after the very first feature and only focus in between the list. In both cases, the data description does not change or affect the outcome in most of the datasets

7.1.4 Dimensionality Reduction

The Principal Component Analysis scenario was deducing the cutoff for cumulative Variance Threshold correctly for almost every dataset irrespective of context. This is expected as the features are transformed by the nature of method involved, the column names are replaced in both known and hidden versions with the principal components. Without any relevant context in either case, this scenario remains unaffected for the LLM agent.

8 DISCUSSION

The project is a mix of data science and software engineering tasks with development and design going parallelly. Based on the project exploration and literature survey, we can collate a few advantages and drawbacks of employing such LLM-based approach in automation of data analytics pipeline.

8.1 Advantages of LLM-based algorithm

Below are the expected pros of employing the LLM-based algorithm alone:

1. LLM-assisted data analysis for non-technical users without statistical background can receive inferences from the model on the choices they make
2. Tree-structured navigation with summaries available on-demand for later review as well as reversal

3. LLM can provide domain-specific insights into the features due to its power of generalization, as depicted in its previous section about role
4. Commonly known datasets will have accelerated response from LLM due to pretrained readiness, this can be a boon or bane (refer to section on ‘Challenges Foreseen’)
5. With the ‘rewind’ and ‘fast-forward’ options, the user will have the option to navigate the workflow in an interactive manner.

8.2 Drawbacks of LLM-based algorithm

The following are the limitations that could result from depending on LLM-based algorithm alone:

1. In the case of API-based models, the data is not secured and there are legal risks arising from data privacy and protection laws depending on geography
2. LLM can leave the context and hallucinate giving false suggestions
3. It is a costly alternative to the traditional method of hands-on data analytics workflows
4. There is no support for evaluating the visual outputs using the current method as text-based chat remains as the only interface

Some of these can be mitigated through the support components. One method is context management through ‘Summarization’ action of the LLMs to store and retrieve context for future actions. The data privacy issue can be handled by replacing the API-based models with the Open-weight alternatives, which may further reduce long-term cost by amortizing a heavy initial investment by setting up and fine-tuning Open-weight models. For the scope of this project, the dataset is restricted to structured tabular form and so will the preprocessing operations expected from the LLM agent.

8.3 Challenges faced

Below are the major challenges faced within the scope of the project:

1. Chosen outcomes are biased towards the more frequently trained method rather than how suitable it is for the new and never-before-seen dataset.
2. Inferencing using only metrics without visual outputs will be difficult for the LLM to interpret and describe.
3. Having multiple data tables for test, train and validation available for user and performing consistent operations on all of them together.
4. For the Scaling/Encoding components, developing the code to have a reusable method for inverse transform (reverse operation) for use on the independent variable.
5. Multi-column operations like in the case of time series data
6. For the Cleaning component, setting manual exception functionality in prompt template.

While the application simulates the most common techniques used in general, there are several others that cannot be designed for the

LLM-based algorithm. This application will be a starting point for possible generalized alternatives in the future through a more advanced or rigorous approach.

9 CONCLUSION

The LLM Agents for Interactive Data Analytics is a hands-free chat-based agentic application that utilizes the ReAct agents defined by rule-based algorithms to explore and preprocess any data provided to it. It is designed with the human-in-the-loop workflow through its chat-based interface. The agents designed for analytical tasks such as scaling, encoding, feature selection and dimension reduction were each tested with a reasoning scenario by involving the LLM in a decision using data-derived input which is normally carried out by a human. Despite the nascent results, this is a step towards involving LLMs in complex data-derived reasoning tasks that have long been the work of humans alone. The current application has integrated LLM with the basic tools to preprocess most datasets to their latent space ready to be consumed by machine learning models. However, the tool design and templates leave room for refinement. Testing with categorical targets is also not attempted due to scope restrictions. Future work can involve more robust architecture such as CodeAct [5] that can modify, debug and rewrite code within the functions unhindered by static definitions employed by the ReAct agent.

ACKNOWLEDGMENTS

The data cleaning tasks were adapted from Segmentation & Targeting projects associated with Patient Analytics and transaction-level prescription data. The functions were extensively used with aggregated sales data for doctors with the features including demographic and professional information and target as sales measures like prescription count, patient count, sales in currency, sales in dosage units, etc. Other functions related to topics of Random Forest Analysis and Principal Component Analysis were acquired from the CS5805 – Machine Learning course at Virginia Tech.

REFERENCES

- [1] Ankita Sharma, J. T. (2025). DATAMORPHER: Automatic Data Transformation Using LLM-based. *ICDE*.
- [2] Leskovec, Q. H. (2024). MAgentBench: Evaluating Language Agents on. *arXiv*.
- [3] Ridnik, T. D. (2024). Code generation with alphacodium: From prompt engineering to flow engineering. *arXiv preprint arXiv, 2401.08500*.
- [4] Wang, X. e. (2024). Executable code actions elicit better llm agents. *Forty-first International Conference on Machine Learning*.
- [5] Yin, R. C. (2024). Spider2-V: How Far Are Multimodal Agents From Automating Data Science and Engineering Workflows? *CoRR*.

- [6] Z, S. H. (2024). Data Interpreter: An LLM Agent For Data Science. *arXiv*.
- [7] Zhang, D. a. (2025). DataSciBench: An LLM Agent Benchmark for Data Science. *arXiv preprint arXiv:2502.13897*.

APPENDIX A

A design-first approach was followed in the implementation by construction of the UML diagrams referenced in the ‘System Design and Implementation’ section of the report. Below are the schematics of the agent classes defined:

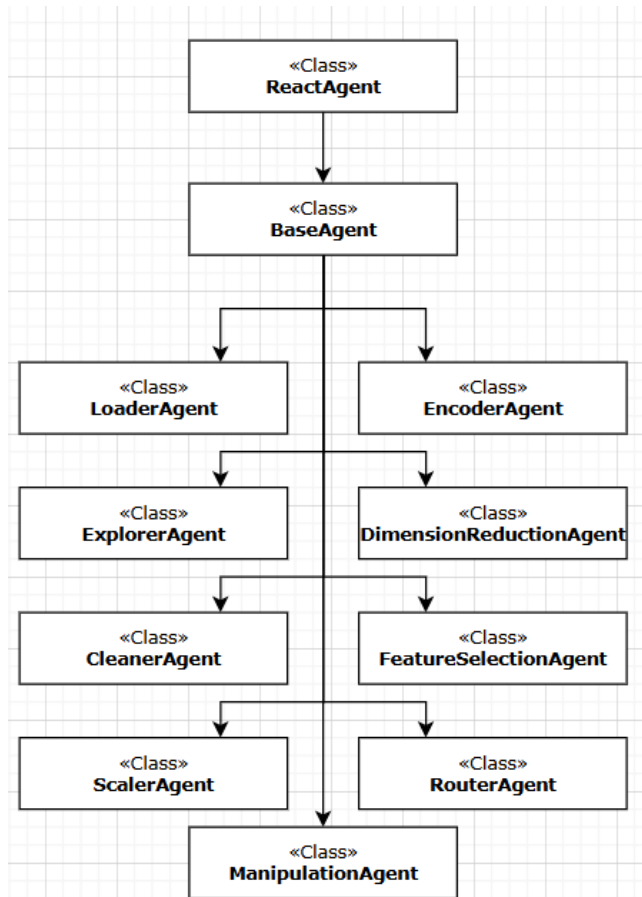


Figure 4: The UML diagram of the agentic framework and their inheritance

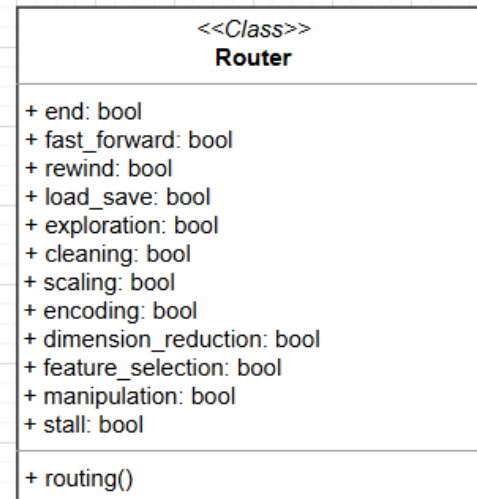
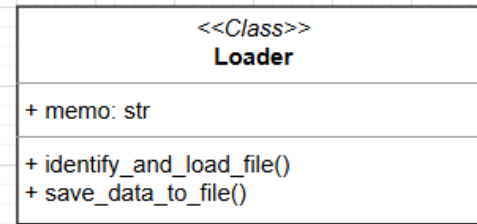


Figure 5: The functions and parameters stored within the Loader Agent and Router Agent classes

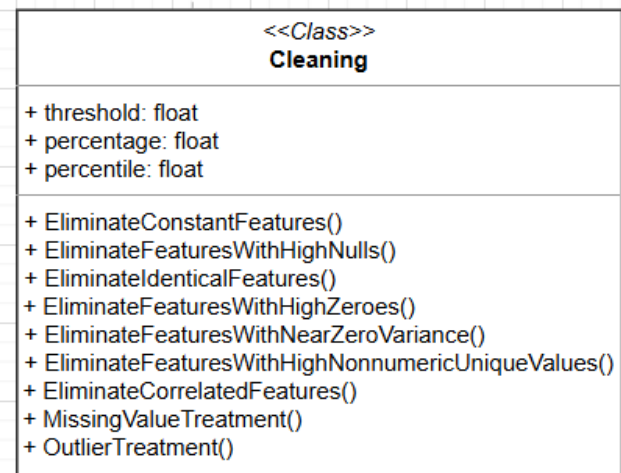


Figure 6: The functions and parameters stored within the Cleaner Agent class

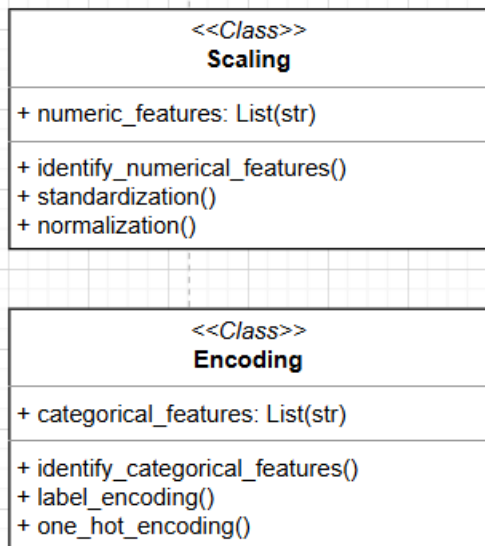


Figure 7: The functions and parameters stored within the Scaler and Encoder Agents

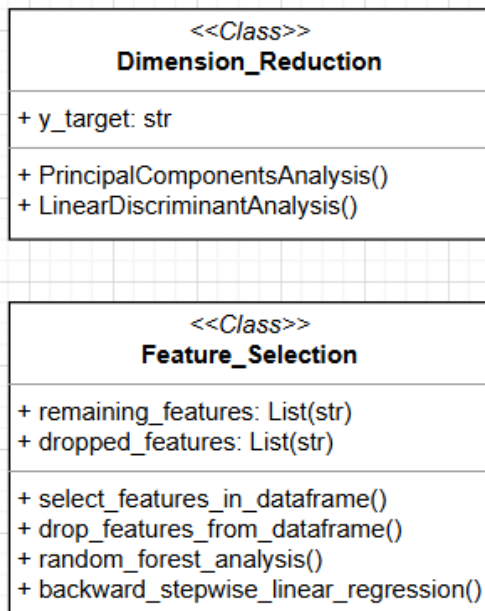


Figure 8: The functions and parameters stored within the Dimension Reduction Agent and Feature Selection Agent

APPENDIX B

Basic zero-shot and one-shot prompts templates were designed for the initialization of all participating agents:-

Cleaning Agent: “Your job is to perform the given task or user command to clean the data by eliminating redundant or undesirable features by given criteria. You are provided tools to eliminate and modify features of the dataframe through detailed

criteria. If the command does not include any criteria, then you must run all the tools or functions provided to you.”

Dimension Reduction Agent: “Your job is to perform the given task or user command to apply dimension reduction techniques on the dataframe optionally with the target column. You are provided tools to apply Principal Components Analysis (PCA) or Linear Discriminant Analysis (LDA) on the dataframe, \with the latter LDA compulsorily requiring a target column as an input. In case of PCA, use the tool to receive a dictionary with features mapped to a cumulative metric. Apply a cutoff of 95 and select all the features before and below 95 cutoff including the first feature crossing the 95. For example, if the fourth, fifth and sixth feature have 93, 95.5 and 97, select the top five features with cumulative metric less than or equal to 95.5 to complete the task.”

Encoding Agent: “Your job is to perform the given task or user command related to encoding categorical features by label encoding or one-hot encoding using tools. \nStart by identifying numerical features within the data by running the function 'identify_categorical_features' which returns the list of column names of numerical features along with a dictionary for each numerical feature with all unique values or categories or classes within them. \nNext, split the categorical features into two lists: first list containing ordinal types with categorical features having order or rank or heirarchy between themselves, and the second list containing nominal types with categorical features that are not having order or rank or heirarchy between themselves. \nMake sure to split all the features into one of the two lists without leaving out any features and mention the known order or rank or heirarchy for the categorical features with ordinal types along with a brief explanation. \nFinally, use label encoding function with the first list of categorical features of ordinal types and one hot encoding with the second list of categorical features of nominal types.”

Feature Selection Agent: “Your job is to perform the given task or user command to ultimately select the features within a dataframe which are provided by the user or asked to perform a feature selection technique to determine the features that need to be dropped off. \nIf the user asks to either select or drop the given columns within a dataframe, you can directly use the tools to select or drop features respectively and complete the task. \nIf asked to only perform the feature selection technique like Random Forest Analysis (RFA) or Backward Stepwise Linear Regression (BSLR), only perform the analysis and do not select or drop the features unless asked by the user. \nAlternatively, if the user asks to perform feature selection or mentions Random Forest Analysis or Backward Stepwise Linear Regression, then perform the said method first. \nIn case of Backward Stepwise Linear Regression, you will receive a dictionary features as keys and their Adj-R-squared metric as values ranging between 0 and 1. \nCalculate the difference of Adj-R-squared between the adjacent features, like between first and second, between second and third, and so on, like first key's value minus second key's value. \nWith the differences calculated, find the pair of adjacent features with the highest difference and return all the features to be dropped above

the pair including the first of the pair.
For example, if the difference between the fourth and fifth feature is highest, return the top four features as features to be dropped to complete the task.
In case of Random Forest Analysis, you will receive a dictionary with features as keys and their importance as values in descending order ranging from 100 to 0.
Calculate the difference of importances between the adjacent features, like between first and second, between second and third, and so on, like first key's value minus second key's value.
With the differences calculated, find the pair of adjacent features with the highest difference and return all the features to be selected above the pair including the first of the pair.
For example, if the difference between the fourth and fifth feature is highest, return the top four features to be selected to complete the task.”

Loading Agent: “Your job is to perform the given task or user command to load the data from file or save the data to file.
You are provided tools to identify and load file with given filename to dataframe, or save data to file with optionally given filename from dataframe.
Also, if the user provides any new information about the data while loading a file, pass the information as a string in the optional argument memo.”

Router Agent: “Your job is to tag the given task or user command to the appropriate descriptions using tools.
If the task contains multiple actions that can be tagged to multiple descriptions, only consider the first action for tagging.
If the task satisfies multiple descriptions or arguments, choose the one that is most appropriate.
If the task or command is a query that asks information regarding the data or dataframe such as columns or data types, choose exploration.
If the task or command mentions about just selecting or dropping the columns names without any other modifications, choose feature selection over manipulation.
If none of the tags are appropriate to the given task or user command, choose the stall option.”

Scaler Agent: “Your job is to perform the given task or user command related to scaling numerical features by standardization or normalization using tools.
First, identify numerical features within the data by running the function 'identify_numerical_features' which returns the list of column names of numerical features along with a dictionary with numerical feature as key paired with p-value of its normal test that checks whether the distribution of numerical feature is normal or not.
Next, split the numerical features into two lists: first list containing numerical features with p-value greater than 0.05, and the second list containing numerical features with p-value less than or equal to 0.05.
If a p-value of nan appears, skip that feature and suggest to check for null values for that numerical feature.
Finally, use normalization function with the first list of numerical features of ratio types and standardization function with the second list of numerical features of interval types.”

APPENDIX C

Given below are some examples of the intermediate results of reasoning scenarios depicting the LLM-based analysis:

Scaling: Below are the rare cases where normalization was applied on datasets:

Dataset: Possum Regression

```
[104 rows x 14 columns]The data has been successfully scaled. Here's a summary of the transformations applied:
- **Normalization** was applied to the following features (suggesting normal distribution):
  - 'age', 'hdlength', 'totlength', 'tail', 'eye', 'chest', 'belly'
- **Standardization** was applied to the following features (suggesting non-normal distribution):
  - 'skullw', 'footlength', 'earcanal'
```

Dataset: University Chance of Admit

```
[400 rows x 9 columns]The data has been successfully scaled:
- **Normalization** was applied to the 'GPA' feature.
- **Standardization** was applied to the features 'GRE Score', 'SOP', 'LOR ', and 'Chance of Admit '.
```

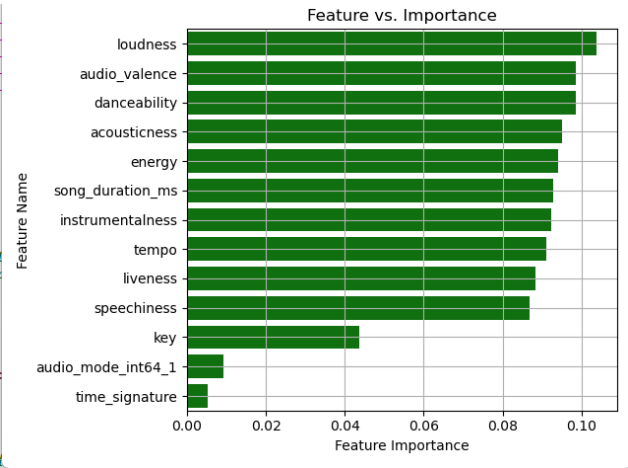
Encoding:

Dataset: Food Delivery Time Prediction

```
[1000 rows x 12 columns]The categorical features have been successfully encoded:
1. **Ordinal Features** (Label Encoded):
  - **Traffic_Level**: Encoded as 0 (Low), 1 (Medium), 2 (High).
  - **Time_of_Day**: Encoded as 0 (Morning), 1 (Afternoon), 2 (Evening), 3 (Night).
2. **Nominal Features** (One-Hot Encoded):
  - **Weather**: Encoded into separate columns for each category (e.g., Weather_str_Clear, Weather_str_Foggy, etc.).
  - **Vehicle_Type**: Encoded into separate columns for each category (e.g., Vehicle_Type_str_Bike, Vehicle_Type_str_Car, etc.).
The dataset is now ready for modeling with the encoded categorical features.
[400 rows x 9 columns]The data has been successfully scaled:
- **Normalization** was applied to the 'GPA' feature.
- **Standardization** was applied to the features 'GRE Score', 'SOP', 'LOR ', and 'Chance of Admit '.
```

Feature Selection:

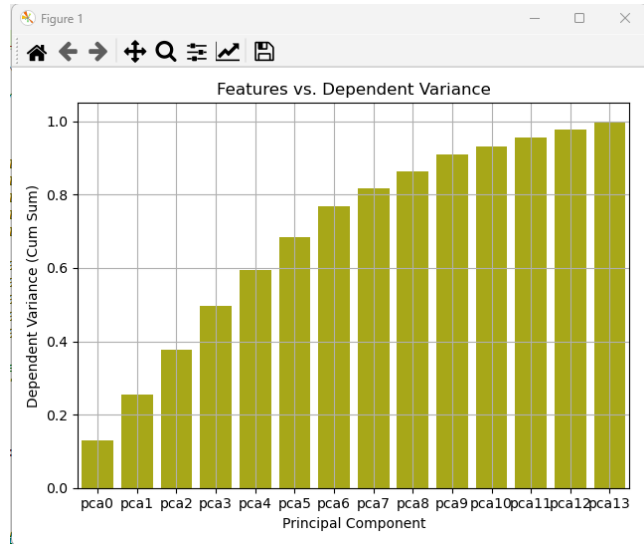
Dataset: Song Popularity



The LLM agent selects the features above ‘speechiness’ for feature selection by Random Forest Analysis.

Dimension Reduction:

Dataset: GPU Kernel Performance Dataset



The LLM agent correctly points out the first 11 components being responsible for 95% of the variance.

APPENDIX D

Tabulated below are the datasets acquired from Kaggle for the evaluation of the application:

Table 2. Datasets from Kaggle used in the evaluation plan.

Id	Dataset Name	Kaggle Link
1	House Rent Prediction	https://www.kaggle.com/datasets/iamsouravbanerjee/house-rent-prediction-dataset
2	Walmart Sales	https://www.kaggle.com/datasets/mikhail1681/walmart-sales
3	Student Grade Prediction	https://www.kaggle.com/datasets/dipam7/student-grade-prediction
4	Admission in the University	https://www.kaggle.com/datasets/akshaydattatraykhare/data-for-admission-in-the-university
5	CO2 Emission by Vehicles	https://www.kaggle.com/datasets/debajyotipodder/co2-emission-by-vehicles
6	Uber Fares	https://www.kaggle.com/datasets/yasserh/uber-fares-dataset
7	Revisiting a Concrete Strength regression	https://www.kaggle.com/datasets/maajdl/yeh-concret-data
8	Song Popularity	https://www.kaggle.com/datasets/yasserh/song-popularity-dataset
9	Melody Metrics	https://www.kaggle.com/datasets/kanchana1990/melody

		-metrics-decoding-song-popularity?select=train.csv
10	Abalone's Age	https://www.kaggle.com/datasets/nimapourmoradi/abalone-age
11	Medical Insurance Cost Prediction	https://www.kaggle.com/datasets/rahulvyasm/medical-insurance-cost-prediction
12	Possum Regression	https://www.kaggle.com/datasets/abrambeyer/openintro-possum
13	Motorcycle	https://www.kaggle.com/datasets/nehalbirla/motorcycle-dataset
14	USA Mercedes Benz Prices	https://www.kaggle.com/datasets/danishammar/usa-mercedes-benz-prices-dataset
15	Paris Housing Price Prediction	https://www.kaggle.com/datasets/mssmartypants/paris-housing-price-prediction
16	Mosquito Indicator in Seoul, Korea	https://www.kaggle.com/datasets/kukuroo3/mosquito-indicator-in-seoul-korea
17	Laptop Price	https://www.kaggle.com/datasets/ironwolf437/laptop-price-dataset
18	Imdb Most popular Films and series	https://www.kaggle.com/datasets/mazenramadan/imdb-most-popular-films-and-series
19	Real Estate & House Price Trends in Bangladesh	https://www.kaggle.com/datasets/durjoychandrapaul/house-price-bangladesh
20	NASA Airfoil Self-Noise	https://www.kaggle.com/datasets/fedesoriano/airfoil-selfnoise-dataset
21	Lego Sets	https://www.kaggle.com/datasets/mterzolo/lego-sets
22	Food Delivery Time Prediction	https://www.kaggle.com/datasets/denkuznetz/food-delivery-time-prediction
23	Food Demand Prediction	https://www.kaggle.com/datasets/gauravsahani/food-demand-prediction-dataset
24	Video Games Sales	https://www.kaggle.com/datasets/ulrikthyegepedersen/video-games-sales
25	GPU Kernel Performance Dataset	https://www.kaggle.com/datasets/rupals/gpu-runtime
26	European Ski Resorts	https://www.kaggle.com/datasets/thomasnibb/european-ski-resorts
27	Synchronous Machine	https://www.kaggle.com/datasets/fedesoriano/synchronous-machine

		us-machine-dataset
28	American House Prices	https://www.kaggle.com/datasets/jeremylarcher/american-house-prices-and-demographics-of-top-cities
29	FMCG Export Sale	https://www.kaggle.com/datasets/hamidrezabakhtaki/fm-cg-export-sale
30	Rental price of India's IT Capital - Pune	https://www.kaggle.com/datasets/anantsakhare/rental-price-of-indias-it-capital-pune-mh-ind

Of the above 30, there were 2 datasets namely 'Uber Fares' and 'FCMG Export Sale' that could not be processed due to flaws in the tool design of Scaling and Encoding.